

Depict Architecture

web version

This document describes the architecture of the web-hosted version of Depict, accessible at <https://hconsult.helioho.st>. It covers how the application is delivered to the browser, how it runs entirely on the user's machine, and how it can optionally be enhanced into a Progressive Web App (PWA) for offline use and home screen installation.

Narrative 1: User Enters the Depict URL in Their Browser

The journey begins the moment a user types `https://hconsult.helioho.st` into their browser's address bar and presses Enter.

DNS Resolution

The browser first needs to find out where `hconsult.helioho.st` lives on the internet. It queries a DNS resolver — usually provided by the user's ISP or a service like Cloudflare — which returns the IP address of the Heliohost server where the Depict application is hosted. This takes milliseconds and is often cached from a previous visit.

TCP Connection and TLS Handshake

The browser opens a TCP connection to that IP address on port 443 (HTTPS). Before any application data flows, the browser and the Heliohost server negotiate a TLS session — they exchange certificates, agree on an encryption cipher, and establish a shared secret key. From this point on, all traffic between the user's machine and the server is encrypted. The user sees the padlock icon in their browser's address bar.

HTTP Request for `index.html`

With the secure connection established, the browser sends an HTTP GET request for `/`. The Depict server — a compiled Rust binary using the Hyper web framework, running as a process on the Heliohost server — receives this request. It has no routing logic to speak of: it simply looks up `/` in its `dist/` directory and serves `index.html`, a small file of around 1,473 bytes. The response comes back with an HTTP 200 status and the HTML content.

Parsing `index.html` and Discovering Assets

The browser parses the HTML. It is a short document generated by Trunk, Depict's build tool. In the `<head>`, the browser finds a set of `<link>` tags that declare everything the page needs before it can run:

- A stylesheet: `index-597d90855083f957.css` (an empty file — the app is currently unstyled)
- Three JavaScript modules via `rel="modulepreload"`: the main `wasm-bindgen` glue file (~41 KB) and two Dioxus runtime snippets (~12 KB and ~0.3 KB)
- The WebAssembly binary via `rel="preload" as="fetch": depict-web-..._bg.wasm` (~4.9 MB)

Every one of these `<link>` tags carries an **integrity** attribute containing a SHA-384 hash. These are Subresource Integrity (SRI) declarations — a security mechanism that tells the browser to refuse execution if the cryptographic fingerprint does not match. Chrome enforces this strictly; Firefox is more lenient.

Parallel Asset Fetching

The browser fires off HTTP requests for all assets in parallel over the same encrypted connection. The Rust server streams them back. The three JS files arrive quickly. The Wasm binary takes a moment longer — at 4.9 MB it is the largest file — and the browser receives it as a stream of 65 KB chunks.

SRI Verification

As each file arrives in full, the browser computes its SHA-384 hash and compares it against the value declared in index.html. If any hash mismatches, the resource is blocked before it can be executed. On Heliohost, all hashes match and all files are accepted.

JavaScript Module Execution

In the <body>, index.html contains a single inline <script type="module">. Once the browser has fetched and verified all the preloaded modules, it executes this script. It imports the wasm-bindgen initialisation function from the JS glue file and calls init(), passing the path to the Wasm binary.

WebAssembly Instantiation

The browser attempts WebAssembly.instantiateStreaming() — a fast path that compiles the Wasm binary while it is still being downloaded. On Heliohost, the Wasm file is not served with the application/wasm MIME type, so the browser falls back to WebAssembly.instantiate(), which waits for the full binary before compiling. Either way, V8 (Chrome) or SpiderMonkey (Firefox) compiles the Wasm bytecode into native machine code on a background thread.

Dioxus Initialisation

With the Wasm module instantiated, the wasm-bindgen glue calls into the Rust code. The Rust main() function runs inside the Wasm sandbox. wasm_logger::init() wires up the Rust logging infrastructure to the browser console. Then Dioxus initialises its virtual DOM, mounts the application component tree onto the <div id="main"> element in the page, and registers event listeners for user interactions. The browser console receives the log messages "Starting up" and "rebuilding app".

The Page is Live

From the user's perspective, the browser tab title updates, the Depict interface appears, and the status indicator shows green: Ready. The total elapsed time from pressing Enter to seeing the interface is typically one to three seconds on a fast connection, dominated by the Wasm download and compilation time. The Rust application is now running natively inside the user's browser tab, in a sandboxed WebAssembly environment, communicating with the DOM through the JavaScript bridge that wasm-bindgen and Dioxus provide.

Narrative 2: User Pastes a Diagram Description and Sees the Result

The application is now live in the user's browser. **Everything that follows happens without any further network requests to the Heliohost server** — Depict is a fully client-side application. The server's job is done.

The User Pastes Input

The user clicks into the text input box and pastes a diagram description written in valid Depict syntax. The browser fires a DOM input event. Dioxus registered a listener for this event during initialisation; the wasm-bindgen bridge catches it on the JavaScript side and calls into the Rust/Wasm code.

State Transition: Ready -> Processing

Inside the Wasm sandbox, the Rust event handler receives the new input text. The Dioxus component updates its state: the application transitions from **Ready** to **Processing**. Dioxus diffs the virtual DOM — it sees that the status indicator component needs to change colour and label — and passes a minimal set of DOM patch instructions to dioxus-interpreter-js, which applies them to the real DOM. The user sees the

indicator turn yellow and the label change to "Processing".

Diagram Parsing

The Rust code parses the pasted text according to Depict's syntax rules. This is pure computation inside the Wasm sandbox — no network, no file I/O, no operating system calls. The parser runs on the user's own CPU, inside the browser process, entirely in memory.

Diagram Rendering

If parsing succeeds, the rendering pipeline runs. The parsed diagram description is transformed into a visual representation — coordinates, shapes, connections, labels. This is again pure Rust computation inside the Wasm sandbox, running at near-native speed because the Wasm bytecode was compiled to native machine code during the instantiation step.

Virtual DOM Update

When rendering completes, the Rust code updates the Dioxus component state with the new diagram output and transitions the application state back to **Ready** (or to **Error** if rendering failed). Dioxus runs its diffing algorithm, computes the minimal set of changes needed to update the displayed diagram, and hands those patches to dioxus-interpreter-js. The interpreter applies them to the real DOM.

The User Sees the Diagram

The browser's rendering engine repaints the affected portion of the page. The status indicator returns to green. The diagram appears in the output area. The entire round-trip — from paste to rendered output — involves no network traffic whatsoever. It is bounded only by the speed of the user's CPU and the complexity of the diagram. Simple diagrams update in milliseconds.

If Something Goes Wrong

If the Depict syntax is invalid, the parser returns an error. The Rust error handler sets the application state to **Error** with an associated message, and transitions the status indicator to red with the label "Failed to render diagram". Dioxus diffs and patches the DOM accordingly. The user sees the red indicator and an explanation of what went wrong. No crash, no page reload — the Wasm sandbox remains live and the user can correct their input and try again.

The Boundary Between the Two Narratives

What makes this architecture notable is where the work splits. Everything in the first narrative — DNS, TCP, TLS, HTTP, file serving — involves multiple physical machines and network infrastructure crossing the public internet. The Heliohost server is doing real work under load.

Everything in the second narrative happens on a single machine: the user's own computer, inside a single browser process, inside a WebAssembly sandbox that has no network access and no file system access. The 4.9 MB Wasm binary that took most of the first narrative to deliver contains the **entire Depict application** — parser, renderer, UI logic — compiled from Rust into a portable binary format that runs identically on Windows, macOS, and Linux, in Chrome, Firefox, and Safari, without any plugins or installations.

Once delivered, Heliohost is completely out of the picture.

Narrative 3: Depict as a Progressive Web App

The two narratives above describe Depict as a conventional web application — loaded fresh from the server each time, living in a browser tab, gone when the tab is closed. A Progressive Web App (PWA) enhancement changes the delivery and installation characteristics of the application without touching the Rust or Wasm code at all. The parser, renderer, and UI logic are identical; only the scaffolding around them changes.

What the PWA Layer Adds

Three additions transform the web app into a PWA: a **Web App Manifest**, a **Service Worker**, and a small **registration script** embedded in index.html. None of these interact with the Wasm sandbox. They operate entirely at the browser infrastructure level, below the application itself.

First Visit: Identical to the Standard Web Version

The user's first visit to <https://hconsult.helioho.st> proceeds exactly as described in Narrative 1. DNS resolution, TLS handshake, asset fetching, SRI verification, Wasm compilation, Dioxus initialisation — everything is the same. The PWA additions are transparent on first load. At the end of that first load, however, two additional things happen that do not occur in the standard web version.

Service Worker Registration

The inline registration script in index.html fires after the page loads:

```
if ('serviceWorker' in navigator) { window.addEventListener('load', () => {  
  navigator.serviceWorker.register('/sw.js'); }); }
```

The browser fetches sw.js from the server and installs it as a background process associated with the origin hconsult.helioho.st. The Service Worker is a JavaScript file that runs in its own thread, separate from the page — it has no access to the DOM and no connection to the Wasm sandbox. Its sole job is to intercept network requests and manage a local cache.

During installation, the Service Worker opens a named cache (depict-v1) and pre-fetches every application asset into it: index.html, the Wasm binary, the JS glue files, the manifest, and the icons. By the time the user has finished their first session, a complete copy of the application is sitting in the browser's local storage, cryptographically identified by URL.

The Install Prompt

Once the browser has verified that the site is served over HTTPS, has a valid Web App Manifest with the required fields, and has a registered Service Worker, it considers the app installable. In Chrome on desktop, an install icon appears in the address bar. On Android Chrome, a banner or menu option offers to add the app to the home screen. On iOS Safari, the user can tap Share and then "Add to Home Screen".

The Web App Manifest declares how the installed app should present itself:

```
{ "name": "Depict", "short_name": "Depict", "start_url": "/", "display": "standalone",  
  "theme_color": "#1a1a2e", "icons": [ { "src": "/icons/icon-192.png", "sizes": "192x192"  
}, { "src": "/icons/icon-512.png", "sizes": "512x512" } ] }
```

The `display: standalone` declaration is what removes the browser chrome — address bar, tabs, navigation buttons — when the app is launched from the home screen or desktop. The installed app appears in its own window, indistinguishable in presentation from a native application.

Subsequent Visits: Served from Cache

When the user returns to Depict — whether by typing the URL, clicking a bookmark, or tapping the installed icon — the Service Worker intercepts the browser's request for index.html before it reaches the network. It finds a cached copy and returns it immediately, without any round-trip to Heliohost.

The same applies to every subsequent asset request. The Wasm binary, all 4.9 MB of it, is served from the local cache in milliseconds rather than downloaded again. The browser still compiles it — that step cannot be skipped — but the compilation can begin immediately rather than waiting for a network download. The overall load time for a return visit drops from one to three seconds to well under a second on most devices.

Offline Operation

If the user opens Depict with no network connection at all — on a plane, on a train, in a building with no signal — the Service Worker cache makes no distinction. The requests are intercepted and served from local storage exactly as they would be with a working network. The application loads, the Wasm compiles and runs, and the user can parse and render diagrams without any awareness that the server is unreachable.

This is a direct consequence of the architecture described in Narrative 2: because Depict performs all of its computation locally and makes no runtime network calls, "offline" is a natural state for it once the assets are cached. The PWA layer simply makes that capability explicit and reliable.

Cache Updates After Deployment

When a new version of Depict is deployed to Heliohost, returning users will not see it immediately. Their Service Worker continues serving the cached previous version. In the background, the browser periodically checks whether `sw.js` has changed. If the file has been updated — specifically, if the `CACHE_NAME` version string has been incremented from `depict-v1` to `depict-v2` — the new Service Worker installs alongside the old one. On the user's next page load after that background check completes, the new Service Worker activates, deletes the old cache, pre-fetches the new assets, and takes over. The user sees the updated version from that point on, with no manual action required.

What the PWA Layer Does Not Change

The PWA additions have no effect on the application's behaviour once it is running. Diagram parsing, rendering, the virtual DOM diffing, the Dioxus component lifecycle — all of this is determined entirely by the Wasm binary and is identical whether the app was loaded from the network, from the Service Worker cache, or from an installed icon. The PWA is a delivery and installation enhancement, not an application change.

Summary of Deployment Variants

Variant	Server at runtime	Offline	Installable	Changes needed
Standard web app	Yes	No	No	None
PWA (cached)	No*	Yes	Yes	Manifest + SW

* After first visit; first load still requires network.